Gridiron GM — Análise do Projeto

Análise técnica e de design do jogo, no estado atual do App.jsx (~2.780 linhas) + customRosters.js.

Visão geral

Gridiron GM é um simulador de gestão de futebol americano no estilo "franchise mode": você assume um time da NFL (com nomes, cidades e cores reais, mas elenco 100% gerado proceduralmente) e toca a carreira — escala o time, define táticas, negocia, faz draft, joga a temporada, disputa playoffs, passa pela offseason e repete. É um app React de arquivo único, mobile-first, com Tailwind via CDN, sem backend: todo o estado vive no localStorage do navegador.

O que mais chama atenção é o quanto o motor de simulação de jogo foi longe. Não é um "RNG rola dado e soma pontos" — é uma engine jogada a jogada, com um playbook de verdade, personnel packages, leitura de blitz, pressão de passe modelada por confronto individual de O-line contra D-line, e por aí vai. Isso é o coração do projeto e também a parte mais sofisticada dele, claramente destacada do resto (telas de gestão, que são mais funcionais que ambiciosas).

Arquitetura técnica

- **Stack**: React (hooks, sem Redux/Context — tudo em useState no componente App), Vite + Tailwind CDN, sem TypeScript.
- **Arquivo único**: App.jsx concentra geração de liga, motor de simulação, lógica de carreira (draft, free agency, trocas, offseason) e ~35 componentes de UI, todos no mesmo arquivo. Funciona, mas é o maior risco estrutural do projeto (ver seção de dívida técnica).
- **customRosters.js**: arquivo separado e documentado, pensado para ser editado por qualquer pessoa (você ou outra) sem tocar no motor — permite sobrescrever jogadores específicos ou elencos inteiros, com preenchimento automático do que não for definido. Boa separação de responsabilidade.
- **Persistência**: localStorage, uma entrada por carreira (nflcareer:<id>), serializando o estado inteiro via JSON.stringify. Sem migração de schema — se a estrutura de um save mudar (e já mudou várias vezes ao longo desta conversa, com campos novos como weekResults, personnel, plays), saves antigos carregam com campos faltando. O código já se defende disso em vários pontos com ?? e fallbacks, mas é um padrão frágil a longo prazo.
- **Determinismo controlado**: PRNG próprio (mulberry32) semeado a partir de strings (seedFromString), usado tanto para geração de liga quanto para simulação de jogos. Isso é uma escolha sólida — dá reprodutibilidade e facilita debugar/testar (foi assim que consegui calibrar as taxas de sack, YPC etc. rodando milhares de simulações fora do navegador).

O motor de simulação

Esta é, de longe, a parte mais trabalhada do projeto, e existem dois motores distintos com propósitos diferentes:

simGameDetailed — usado no jogo do usuário e nos jogos de playoff que o envolvem. Simula descida por descida, com relógio de verdade, quarters, intervalo, timeouts (3 por tempo, usados automaticamente pelo time que está perdendo no fim do jogo), aviso de dois minutos, e overtime. Cada jogada de passe escolhe um **personnel package** (11/12/21/10, com percentuais configuráveis nas Táticas), decide se vai para **formação vazia** (RB sai para rota em vez de bloquear), roda uma leitura de **blitz** (SAM/WILL, com o RB tentando segurar o rush quando não está em formação vazia), calcula **proteção de passe** por confronto individual (LT x rusher da esquerda, RT x da direita,

interior x melhor rusher interno) que decide entre bolso limpo, hurry, QB hit ou sack — com o jogador específico credenciado nas estatísticas — e só então escolhe um **conceito de rota** (10 rotas nomeadas, cada uma com alvo, defensor responsável e faixa de profundidade) ponderado pela situação de down-and-distance. A corrida usa gaps nomeados (5 buracos, cada um com o lineman responsável) cujo confronto de bloqueio entra na conta da jarda ganha. A chamada de jogada (run/pass ratio) reage à situação de placar e tempo restante — time perdendo por duas posses no 4º quarto passa muito mais, time ganhando corre para gastar relógio.

quickGameStats — usado para os outros 15 jogos da rodada que o usuário não está assistindo. É uma aproximação estatística (sem jogada a jogada) calibrada para produzir distribuições parecidas com as do motor detalhado — mesma faixa de líder de tackles, mesmo teto de jardas corridas, mesma proporção de pressão/sack — para que os líderes da liga não fiquem artificialmente distorcidos entre "seu time" (motor rico) e "resto da liga" (motor simples).

Eu validei boa parte disso rodando simulações em lote fora do navegador (via Node, bundlando o arquivo com esbuild) em vários pontos da conversa. Os números batem com faixas reais de NFL: ~65-69% de completude, ~6-8% de taxa de sack, ~5,1 jardas por corrida, líder de tackles tipicamente 10-14 por jogo, taxa de pressão total ~30-38%. Isso não é trivial de acertar num motor procedural e foi o resultado de várias rodadas de calibração.

Onde o motor ainda é simplificado

- Não existe jogador "cansado" ao longo do jogo (fadiga só existe como conceito narrativo no slider de rotação de RB, não afeta atributos em tempo real).
- O QB não tem mobilidade própria — não há scramble; sob pressão ele só sofre hurry/hit/sack ou arremessa.
- As jardas por corrida (~5,1) ainda ficam um pouco acima da média real da NFL (~4,2-4,5); ficou assim de propósito por já ter sido ajustado uma vez a pedido, mas é um número que dá para apertar mais se quiser.
- INT rate (~0,7-1,3% dos passes) está abaixo do real (~2,2%) — um ponto que ficou pendente de uma calibração dedicada.
- quickGameStats não tem jogada a jogada, então a lógica situacional de placar/tempo (que é bem legal no motor detalhado) não existe para os outros 15 jogos da rodada.

Sistema de carreira e gestão

- **Elenco**: geração procedural por tier (star/starter/backup/bench) com atributos derivados do overall + variância, mais o hook de customRosters.js para sobrescrever. Cada posição tem exatamente 2 sub-atributos (ex.: QB → ARM/ACC, RB → SPD/POW), o que mantém a interface simples mas também limita a profundidade de simulação por jogador.
- **Escalação (Depth Chart)**: reordenação por toque, mais um diagrama 2D de formação (ataque e defesa) que desenha os titulares nas posições reais do campo — incluindo ajuste automático para esquema 4-3 vs 3-4.
- **Táticas**: passe x corrida, agressividade em 4ª descida, rotação de RB2, mix de personnel, % de formação vazia — tudo alimenta o motor detalhado diretamente.
- **Temporada**: calendário round-robin gerado por circleSchedule, 17 semanas, aplicação de resultado (applyResult), tela de Resultados navegável por semana com box score reconstruído via *snapshot/diff* das estatísticas de temporada (technique elegante: em vez de guardar um box score separado, ele tira uma foto das stats antes e depois do jogo e calcula a diferença).
- **Playoffs**: chaveamento por conferência com seeds, simulados rodada a rodada.

- **Offseason**: aposentadoria/substituição automática por idade e overall (retireAndRefreshLeague), draft e free agency.
- **Trocas e Free Agency**: aqui está a maior lacuna de "vida própria" da liga — **só o seu time troca e assina jogadores**. Os outros 31 times não competem pelo mercado de free agency, não fazem propostas de troca, e no draft apenas pegam o próximo prospect da lista (sem noção de necessidade posicional). Eles evoluem só via aposentadoria automática. Isso é a diferença mais perceptível entre "meu time" e "o resto da liga" em termos de gestão (no motor de simulação de jogo, por outro lado, o resto da liga já está bem mais nivelado, como descrito acima).
- **Pressão do dono/torcida**: ownerConfidence/fanSupport existem e te demitem se a confiança chegar a zero, mas hoje só se movem através das respostas de coletiva de imprensa (PRESS_EVENTS) — não reagem diretamente a sequências de vitórias/derrotas ou à posição na classificação, o que é uma desconexão entre "como você está jogando" e "se você está sendo cobrado por isso".

Visual e experiência

O trabalho visual recente elevou bastante a percepção de polimento:

- **Escudos gerados em SVG** por time (gradiente nas cores reais + sigla), sem depender de logos licenciados — solução inteligente para o problema de direito autoral de marcas da NFL.
- **Minicampo 2D** durante o jogo ao vivo, mostrando direção e resultado de cada jogada revelada (corrida/passe/sack/interceptação/touchdown, com cor e seta), sincronizado com o replay incremental do log.
- **Diagrama de formação** na Escalação, com posicionamento realista dos titulares.
- **Páginas de detalhe de time e jogador**, abertas por clique a partir de várias telas (Painel, Elenco, Classificação, Resultados, Stats), com navegação de "voltar" que lembra a tela de origem mesmo em cadeias time→jogador→time.
- Interface consistente por Card/Btn/Bar/PosPill reutilizados, dark theme, mobile-first genuíno (bottom nav, layout de coluna única).

Pontos ainda não cobertos pelo polimento visual: Trocas, Free Agency e Draft continuam com a UI mais antiga/funcional, sem os escudos e sem abrir as páginas de detalhe ao clicar em nomes — ficou assim por decisão explícita de escopo em rodadas anteriores, não por esquecimento.

Pontos fortes

1. **O motor de simulação é o diferencial real do projeto.** Muita gente que faz "GM sim" caseiro para no placar aleatório; aqui existe playbook, matchups individuais e uma leitura de situação de jogo que dá muito mais textura ao replay.
2. **Calibração levada a sério.** Os números de completude, sack, YPC etc. foram checados contra referências reais, não só "parecem certos".
3. **Extensibilidade pensada de propósito**: customRosters.js é um bom exemplo de dar ao usuário um ponto de extensão sem expor a complexidade do motor.
4. **Estatísticas avançadas de verdade**: QB hits, hurries, pressões sofridas, sacks sofridos, % de duelos de bloqueio vencidos, TFL — é raro um jogo caseiro chegar a esse nível de granularidade nem de longe.

5. **Mobile-first genuíno**, não só responsivo — o formato de tela, a navegação por abas embaixo e o tamanho dos elementos foram pensados para uso em celular.

Limitações e dívida técnica

1. **Arquivo único de ~2.780 linhas.** Funciona hoje, mas cada mudança nova aumenta o custo de navegar e de evitar colisão entre features (já aconteceu nesta conversa: uma edição via str_replace sumir com uma linha sem querer por causa de um bloco de texto ambíguo). Separar em módulos (engine/, screens/, data/) em algum momento reduziria esse risco.
2. **Zero testes automatizados.** Toda a validação de calibração que fizemos ao longo da conversa foi manual, via scripts descartáveis rodados fora do app. Não existe nenhum teste que trave, por exemplo, "a taxa de sack não pode passar de X%" — se um ajuste futuro desequilibrar isso, só se descobre jogando.
3. **IA do resto da liga é estática fora do jogo em si.** Como já descrito: sem free agency, sem trocas, sem draft com critério, times só mudam por aposentadoria. Isso não afeta a qualidade da simulação de jogo, mas limita o quanto a liga "vive" ao longo de uma carreira longa.
4. **Sem schema versionado no save.** Cada campo novo depende de fallback manual (??) espalhado pelo código; não há uma rotina central de migração de save antigo.
5. **Sem separação entre lógica pura e componentes.** Funções de simulação, cálculo de cap, geração de liga etc. estão no mesmo arquivo que os componentes React — dificulta reaproveitar a lógica fora do app (ex.: um script de simulação em lote) sem duplicar ou depender do bundler.

Sugestões de próximos passos

Nenhuma delas é urgente — o app está em um estado bom e jogável — mas por ordem de impacto prático:

1. **Levar o polimento visual (escudos + páginas de detalhe) para Trocas, Free Agency e Draft.** É o gap mais visível hoje entre telas.
2. **Dar alguma vida à IA da liga**: mesmo uma versão simples — times cortam o pior jogador de uma posição fraca e assinam um free agent aleatório de tier compatível, ou fazem trocas ocasionais por necessidade posicional — já mudaria a sensação de "liga viva" em carreiras longas de vários anos.
3. **Conectar confiança do dono/torcida ao desempenho real** (sequência de vitórias/derrotas, posição na classificação), não só às respostas de coletiva.
4. **Fechar os dois números de calibração pendentes**: taxa de interceptação um pouco baixa e YPC um pouco alto.
5. **Extrair a "engine" (simulação + geração de liga) para um arquivo/módulo separado dos componentes de UI**, mesmo mantendo tudo num único projeto — reduz risco de edição e abre caminho para testes automatizados básicos (ex.: rodar 1.000 jogos e checar se as taxas ficam dentro de uma faixa esperada, travando regressões).
6. **Migração de save simples**: uma função migrateSave(raw) central que preenche campos novos ausentes, chamada uma vez no load, em vez de fallbacks espalhados pelo código.

Números do projeto

Linhas em App.jsx	~2.780
Linhas em customRosters.js	61
Componentes de tela	~20
Conceitos de rota no playbook	10
Personnel packages	4 (11 / 12 / 21 / 10)
Categorias de stat na liderança da liga	19
Times	32, gerados proceduralmente (com hook de customização)